

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών
1η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614
Φίλιππος Γιαννακόπουλος 03121629
Ομάδα 17

Ζήτημα 1.1

Το Ζήτημα 1.1 βρίσκεται στο αρχείο `erg1.1.asm`.

```
.include "m328Pbdef.inc"

.equ FOSC_MHZ=16
.equ DEL_mS=1000      ; the amount of milliseconds to wait
.equ F1=FOSC_MHZ*DEL_mS

reset:
    ; Initialize Stack
    ldi r24, LOW(RAMEND)
    out SPL, r24
    ldi r24, HIGH(RAMEND)
    out SPH, r24

main:
    ldi r24, low(F1)
    ldi r25, high(F1)
    rcall wait_x_msec
    rjmp finish

; wait_x_msec generates a delay of (6 + 1000 * x + 12 = 1000*x + 18) cycles
wait_x_msec:
    push r23      ; 2 cycles
    push r24      ; 2 cycles
    push r25      ; 2 cycles
repeat_x:        ; ! (x-1) * (996 + 2 + 2) + 996 + 2 + 1 + 10 = 1000 * x + 13 - 1 = 1000*x + 12 !
    rcall wait_one_msec ; 3 + 993 = 996 cycles
    sbiw r24,1      ; 2 cycles
    brne repeat_x   ; 1 or 2 cycles

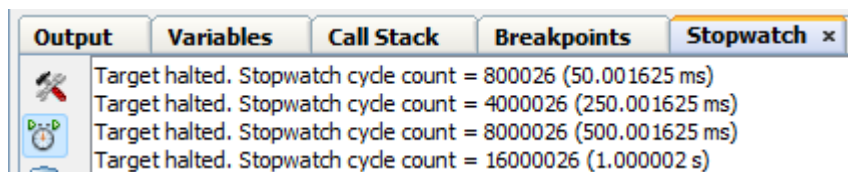
    pop r25         ; 2 cycles
    pop r24         ; 2 cycles
    pop r23         ; 2 cycles
    ret             ; 4 cycles

wait_one_msec:    ; ! 246 * 4 + 8 + 1 = 993 cycles *
    ldi r23, 247   ; 1 cycle
repeat_one:      ; ! 4 cycles (last 3 + 5 = 8 cycles) !
    dec r23        ; 1 cycle
    nop            ; 1 cycle
    brne repeat_one ; 1 or 2 cycles

    nop            ; 1 cycle
    ret            ; 4 cycles

finish:
```

Με μερικές δοκιμές σε χρόνους 50ms, 250ms, 500ms και 1000ms:



Παρατηρούμε σταθερά ένα πλεόνασμα 26 κύκλων εκ των οποίων 18 ανήκουν στην ρουτίνα `wait_x_msec`.

Ζήτημα 1.2

Το Ζήτημα 1.2 βρίσκεται στο αρχείο erg1.2.asm.

```
.include "m328Pbdef.inc"
```

```
.def A=r20  
.def B=r21  
.def C=r22  
.def D=r23  
.def Bt=r24  
.def Ct=r25  
.def Dt=r26  
.def F0=r27  
.def F1=r28  
.def temp=r29
```

```
.def counter=r30
```

```
reset:
```

```
    ; Initialize Stack  
    ldi r24, LOW(RAMEND)  
    out SPL, r24  
    ldi r24, HIGH(RAMEND)  
    out SPH, r24
```

```
main:
```

```
    ldi A,0x51  
    ldi B,0x41  
    ldi C,0x21  
    ldi D,0x01  
    ldi counter,0x06
```

```
repeat:
```

```
    ; Copy the registers  
    mov Bt,B  
    mov Ct,C  
    mov Dt,D  
    ; Inverse them  
    com Bt  
    com Ct  
    com Dt  
    ; F0  
    mov temp,A  
    and temp,Bt  
    mov F0,Bt  
    and F0,D  
    or F0,temp  
    com F0
```

```
    ; F1  
    mov temp,A  
    or temp,Ct  
    mov F1,B  
    or F1,Dt  
    and F1,temp
```

```
    ; Prepare new ABCD  
    ldi temp,0x01  
    add A,temp  
    inc temp  
    add B,temp
```

```

inc temp
add C,temp
inc temp
add D,temp

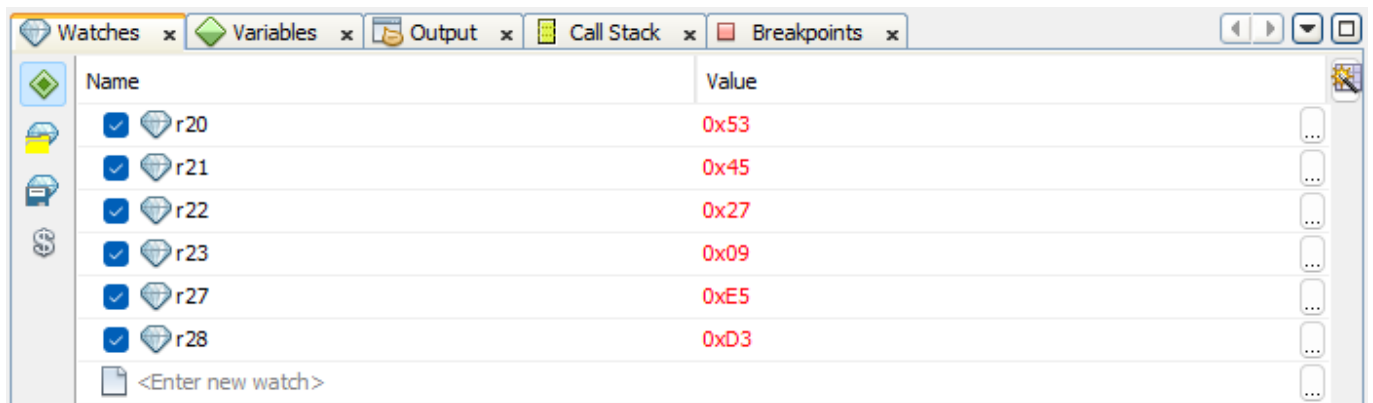
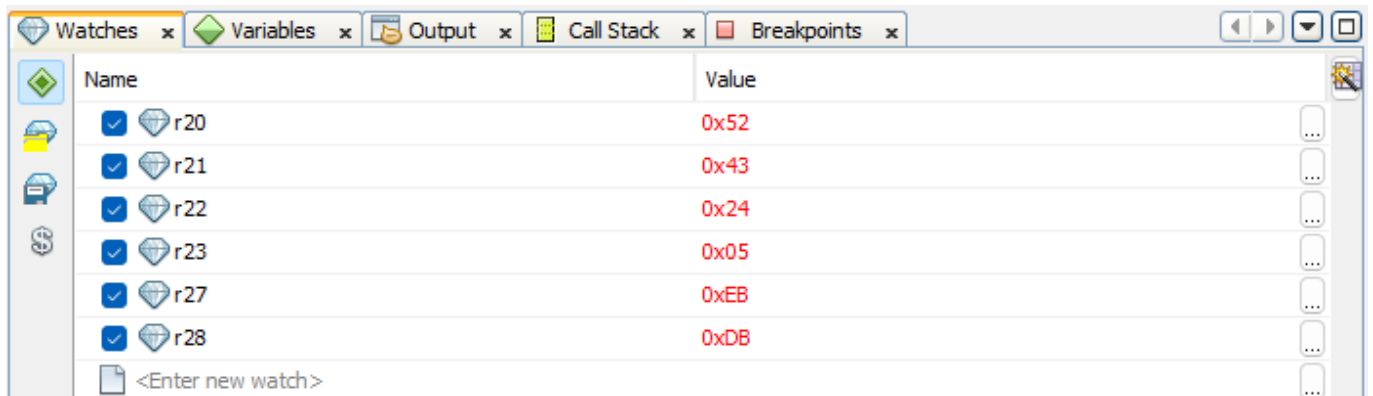
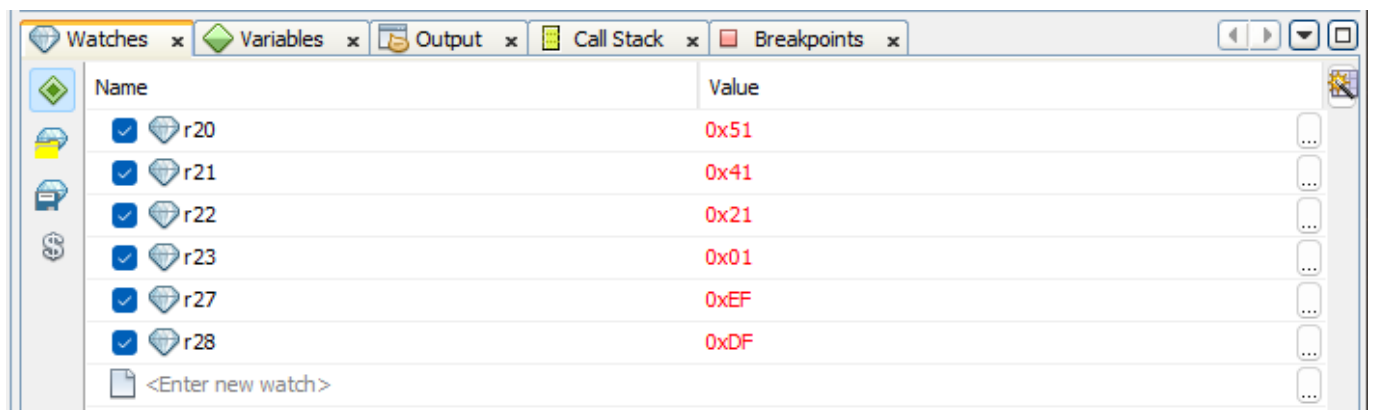
dec counter
brne repeat

```

Τα αποτελέσματα είναι:

A	B	C	D	F0	F1
0x51	0x41	0x21	0x01	0xEF	0xDF
0x52	0x43	0x24	0x05	0xEB	0xDB
0x53	0x45	0x27	0x09	0xE5	0xD3
0x54	0x47	0x2A	0x0D	0xE7	0xD5
0x55	0x49	0x2D	0x11	0xEB	0xC7
0x56	0x4B	0x30	0x15	0xEB	0xCB

Τα οποία αναχτήθηκαν με χρήση Watchers και Breakpoint:



Watches x Variables x Output x Call Stack x Breakpoints x		
Name	Value	
☒ r20	0x54	...
☒ r21	0x47	...
☒ r22	0x2A	...
☒ r23	0x0D	...
☒ r27	0xE7	...
☒ r28	0xD5	...
<Enter new watch>		

Watches x Variables x Output x Call Stack x Breakpoints x		
Name	Value	
☒ r20	0x55	...
☒ r21	0x49	...
☒ r22	0x2D	...
☒ r23	0x11	...
☒ r27	0xEB	...
☒ r28	0xC7	...
<Enter new watch>		

Watches x Variables x Output x Call Stack x Breakpoints x		
Name	Value	
☒ r20	0x56	...
☒ r21	0x4B	...
☒ r22	0x30	...
☒ r23	0x15	...
☒ r27	0xEB	...
☒ r28	0xCB	...
<Enter new watch>		

Όπου:

- r20 αντιστοιχεί στο A
- r21 αντιστοιχεί στο B
- r22 αντιστοιχεί στο C
- r23 αντιστοιχεί στο D
- r27 αντιστοιχεί στο F0
- r28 αντιστοιχεί στο F1

Ζήτημα 1.3

Το Ζήτημα 1.3 βρίσκεται στο αρχείο `erg1.3.asm`.

```
.include "m328Pbdef.inc"

.equ FOSC_MHZ=16
.equ DEL_mS=1000
.equ F1=FOSC_MHZ*DEL_mS

.def pos=r31

.org 0x0
    rjmp reset

reset:
    ; Initialize Stack Pointer
    ldi r24, LOW(RAMEND)
    out SPL, r24
    ldi r24, HIGH(RAMEND)
    out SPH, r24

    ; Load delay to r24-r25
    ldi r24, low(F1)
    ldi r25, high(F1)

    ldi pos,0b10000000

    ser r24          ; r24 = 0xFF
    out DDRD,r24     ; Set DDRD as output

    out PORTD,pos    ; Output initial position.
    set              ; T=1, we're moving to the left

    rcall wait_x_msec

to_left:
    lsr pos
    out PORTD,pos

    rcall wait_x_msec ; 1s delay
    sbrs pos,0        ; Skip if we're in the first bit (bit 0 = 1)
    rjmp to_left

    set                ; T=0
    rcall wait_x_msec
    rjmp to_right      ; Change_direction

to_right:
    lsl pos            ; Shift bit to the right, left on the LEDs
    out PORTD,pos      ; output position

    rcall wait_x_msec ; 1s delay
    sbrs pos,7         ; Skip if we're in the last bit (bit 7 = 1)
    rjmp to_right

    clt                ; T=0
    rcall wait_x_msec
    rjmp to_left       ; Change_direction

wait_x_msec:
    push r23           ; 2 cycles
```

```

    push r24      ; 2 cycles
    push r25      ; 2 cycles
repeat_x:        ; ! (x-1) * (996 + 2 + 2) + 996 + 2 + 1 + 10 = 1000 * x + 13 - 1 = 1000*x + 12 !
    rcall wait_one_msec ; 3 + 993 = 996 cycles
    sbiw r24,1    ; 2 cycles
    brne repeat_x ; 1 or 2 cycles

    pop r25       ; 2 cycles
    pop r24       ; 2 cycles
    pop r23       ; 2 cycles
    ret           ; 4 cycles

wait_one_msec:   ; ! 246 * 4 + 8 + 1 = 993 cycles *
    ldi r23, 247 ; 1 cycle
repeat_one:     ; ! 4 cycles (last 3 + 5 = 8 cycles) !
    dec r23     ; 1 cycle
    nop         ; 1 cycle
    brne repeat_one ; 1 or 2 cycles

    nop         ; 1 cycle
    ret         ; 4 cycles

```

Σε δοκιμές και στην πλακέτα αλλά και στον Simulator (με αλλαγμένη χρονοκαθυστέρηση σε 100ms έτσι ώστε να αντιστοιχεί σε πραγματικό χρόνο περίπου του 1s), παρατηρούμε ότι η λειτουργία της άσκησης γίνεται κανονικά.

Ο έλεγχος στον Simulator έγινε με χρήση Runtime Watch στην μεταβλητή PORTD με μορφή προβολής Binary.